

✓ Máy Vectơ Hỗ trợ (Support Vector Machines)

Sổ tay này chứa tất cả mã mẫu và lời giải cho các bài tập trong chương 5.

 [Open in Colab](#)  [Open in Kaggle](#)

✓ Thiết lập (Setup)

Dự án này yêu cầu Python 3.7 trở lên:

```
import sys

assert sys.version_info >= (3, 7)
```

Nó cũng yêu cầu Scikit-Learn $\geq 1.0.1$:

```
from packaging import version
import sklearn

assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

Như chúng ta đã làm trong các chương trước, hãy xác định kích thước phông chữ mặc định để các hình vẽ trông đẹp hơn:

```
import matplotlib.pyplot as plt

plt.rc('font', size=14)
plt.rc('axes', labelsz=14, titlesz=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsz=10)
plt.rc('ytick', labelsz=10)
```

Chúng ta hãy tạo thư mục (nếu chưa tồn tại) và định nghĩa hàm `save_fig()` được sử dụng trong suốt sổ tay này để lưu các hình vẽ ở độ phân giải cao cho cuốn sách:

```
from pathlib import Path

IMAGES_PATH = Path() / "images" / "svm"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

✓ Phân loại SVM Tuyến tính (Linear SVM Classification)

Cuốn sách bắt đầu với một vài hình vẽ trước ví dụ mã đầu tiên, vì vậy ba ô tiếp theo sẽ tạo và lưu các hình này. Bạn có thể bỏ qua chúng nếu muốn.

```
# extra code - this cell generates and saves Figure 5-1

import matplotlib.pyplot as plt
import numpy as np
from sklearn.svm import SVC
from sklearn import datasets

iris = datasets.load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris.target
```

```

setosa_or_versicolor = (y == 0) | (y == 1)
X = X[setosa_or_versicolor]
y = y[setosa_or_versicolor]

# SVM Classifier model
svm_clf = SVC(kernel="linear", C=1e100)
svm_clf.fit(X, y)

# Bad models
x0 = np.linspace(0, 5.5, 200)
pred_1 = 5 * x0 - 20
pred_2 = x0 - 1.8
pred_3 = 0.1 * x0 + 0.5

def plot_svc_decision_boundary(svm_clf, xmin, xmax):
    w = svm_clf.coef_[0]
    b = svm_clf.intercept_[0]

    # At the decision boundary, w0*x0 + w1*x1 + b = 0
    # => x1 = -w0/w1 * x0 - b/w1
    x0 = np.linspace(xmin, xmax, 200)
    decision_boundary = -w[0] / w[1] * x0 - b / w[1]

    margin = 1/w[1]
    gutter_up = decision_boundary + margin
    gutter_down = decision_boundary - margin
    svcs = svm_clf.support_vectors_

    plt.plot(x0, decision_boundary, "k-", linewidth=2, zorder=-2)
    plt.plot(x0, gutter_up, "k--", linewidth=2, zorder=-2)
    plt.plot(x0, gutter_down, "k--", linewidth=2, zorder=-2)
    plt.scatter(svs[:, 0], svcs[:, 1], s=180, facecolors='AAA',
                zorder=-1)

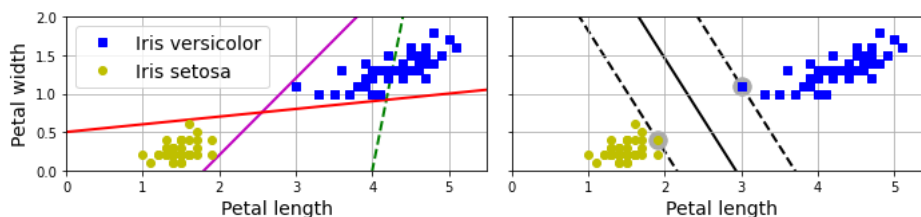
fig, axes = plt.subplots(ncols=2, figsize=(10, 2.7), sharey=True)

plt.sca(axes[0])
plt.plot(x0, pred_1, "g--", linewidth=2)
plt.plot(x0, pred_2, "m-", linewidth=2)
plt.plot(x0, pred_3, "r-", linewidth=2)
plt.plot(X[:, 0][y==1], X[:, 1][y==1], "bs", label="Iris versicolor")
plt.plot(X[:, 0][y==0], X[:, 1][y==0], "yo", label="Iris setosa")
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.legend(loc="upper left")
plt.axis([0, 5.5, 0, 2])
plt.gca().set_aspect("equal")
plt.grid()

plt.sca(axes[1])
plot_svc_decision_boundary(svm_clf, 0, 5.5)
plt.plot(X[:, 0][y==1], X[:, 1][y==1], "bs")
plt.plot(X[:, 0][y==0], X[:, 1][y==0], "yo")
plt.xlabel("Petal length")
plt.axis([0, 5.5, 0, 2])
plt.gca().set_aspect("equal")
plt.grid()

save_fig("large_margin_classification_plot")
plt.show()

```



extra code - this cell generates and saves Figure 5-2

```
from sklearn.preprocessing import StandardScaler
```

```
Xs = np.array([[1, 50], [5, 20], [3, 80], [5, 60]]).astype(np.float64)
```

```

ys = np.array([0, 0, 1, 1])
svm_clf = SVC(kernel="linear", C=100).fit(Xs, ys)

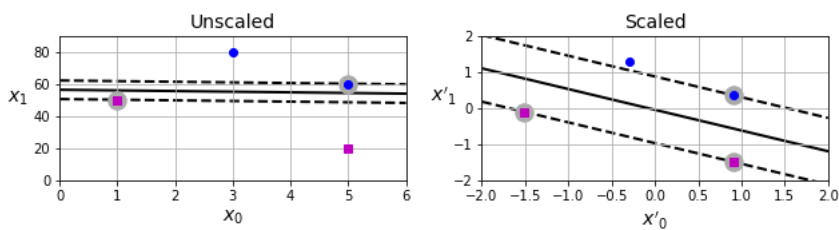
scaler = StandardScaler()
X_scaled = scaler.fit_transform(Xs)
svm_clf_scaled = SVC(kernel="linear", C=100).fit(X_scaled, ys)

plt.figure(figsize=(9, 2.7))
plt.subplot(121)
plt.plot(Xs[:, 0][ys==1], Xs[:, 1][ys==1], "bo")
plt.plot(Xs[:, 0][ys==0], Xs[:, 1][ys==0], "ms")
plot_svc_decision_boundary(svm_clf, 0, 6)
plt.xlabel("$x_0$")
plt.ylabel("$x_1$ ", rotation=0)
plt.title("Unscaled")
plt.axis([0, 6, 0, 90])
plt.grid()

plt.subplot(122)
plt.plot(X_scaled[:, 0][ys==1], X_scaled[:, 1][ys==1], "bo")
plt.plot(X_scaled[:, 0][ys==0], X_scaled[:, 1][ys==0], "ms")
plot_svc_decision_boundary(svm_clf_scaled, -2, 2)
plt.xlabel("$x'_0$")
plt.ylabel("$x'_1$ ", rotation=0)
plt.title("Scaled")
plt.axis([-2, 2, -2, 2])
plt.grid()

save_fig("sensitivity_to_feature_scales_plot")
plt.show()

```



✎ Phân loại Biên mềm (Soft Margin Classification)

```

# extra code - this cell generates and saves Figure 5-3

X_outliers = np.array([[3.4, 1.3], [3.2, 0.8]])
y_outliers = np.array([0, 0])
Xo1 = np.concatenate([X, X_outliers[:, 1]], axis=0)
yo1 = np.concatenate([y, y_outliers[:, 1]], axis=0)
Xo2 = np.concatenate([X, X_outliers[:, 1]], axis=0)
yo2 = np.concatenate([y, y_outliers[:, 1]], axis=0)

svm_clf2 = SVC(kernel="linear", C=10**9)
svm_clf2.fit(Xo2, yo2)

fig, axes = plt.subplots(ncols=2, figsize=(10, 2.7), sharey=True)

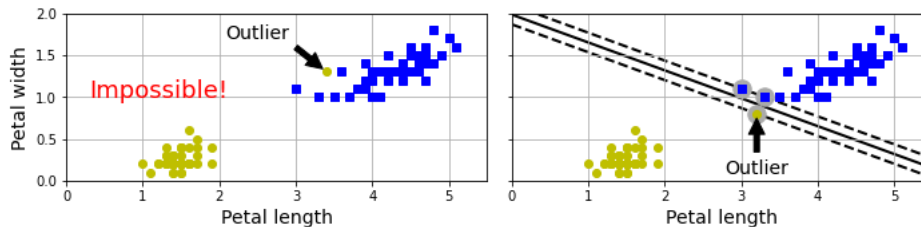
plt.sca(axes[0])
plt.plot(Xo1[:, 0][yo1==1], Xo1[:, 1][yo1==1], "bs")
plt.plot(Xo1[:, 0][yo1==0], Xo1[:, 1][yo1==0], "yo")
plt.text(0.3, 1.0, "Impossible!", color="red", fontsize=18)
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.annotate(
    "Outlier",
    xy=(X_outliers[0][0], X_outliers[0][1]),
    xytext=(2.5, 1.7),
    ha="center",
    arrowprops=dict(facecolor='black', shrink=0.1),
)
plt.axis([0, 5.5, 0, 2])
plt.grid()

plt.sca(axes[1])

```

```
plt.plot(Xo2[:, 0][yo2==1], Xo2[:, 1][yo2==1], "bs")
plt.plot(Xo2[:, 0][yo2==0], Xo2[:, 1][yo2==0], "yo")
plot_svc_decision_boundary(svm_clf2, 0, 5.5)
plt.xlabel("Petal length")
plt.annotate(
    "Outlier",
    xy=(X_outliers[1][0], X_outliers[1][1]),
    xytext=(3.2, 0.08),
    ha="center",
    arrowprops=dict(facecolor='black', shrink=0.1),
)
plt.axis([0, 5.5, 0, 2])
plt.grid()

save_fig("sensitivity_to_outliers_plot")
plt.show()
```



Lưu ý: Giá trị mặc định cho siêu tham số `dual` của `LinearSVC` và `LinearSVR` sẽ thay đổi từ `True` sang `"auto"` trong Scikit-Learn 1.4, vì vậy tôi đã thiết lập `dual=True` xuyên suốt số tay này để đảm bảo kết quả không thay đổi.

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 2) # Iris virginica

svm_clf = make_pipeline(StandardScaler(),
                        LinearSVC(C=1, dual=True, random_state=42))
svm_clf.fit(X, y)
```

```
Pipeline(steps=[('standardscaler', StandardScaler()),
                 ('linearsvc', LinearSVC(C=1, random_state=42))])
```

```
X_new = [[5.5, 1.7], [5.0, 1.5]]
svm_clf.predict(X_new)
```

```
array([ True, False])
```

```
svm_clf.decision_function(X_new)
```

```
array([ 0.66163411, -0.22036063])
```

extra code - this cell generates and saves Figure 5-4

```
scaler = StandardScaler()
svm_clf1 = LinearSVC(C=1, max_iter=10_000, dual=True, random_state=42)
svm_clf2 = LinearSVC(C=100, max_iter=10_000, dual=True, random_state=42)

scaled_svm_clf1 = make_pipeline(scaler, svm_clf1)
scaled_svm_clf2 = make_pipeline(scaler, svm_clf2)

scaled_svm_clf1.fit(X, y)
scaled_svm_clf2.fit(X, y)

# Convert to unscaled parameters
b1 = svm_clf1.decision_function([-scaler.mean_ / scaler.scale_])
b2 = svm_clf2.decision_function([-scaler.mean_ / scaler.scale_])
w1 = svm_clf1.coef_[0] / scaler.scale_
w2 = svm_clf2.coef_[0] / scaler.scale_
svm_clf1.intercept_ = np.array([b1])
```

```

svm_clf2.intercept_ = np.array([b2])
svm_clf1.coef_ = np.array([w1])
svm_clf2.coef_ = np.array([w2])

# Find support vectors (LinearSVC does not do this automatically)
t = y * 2 - 1
support_vectors_idx1 = (t * (X.dot(w1) + b1) < 1).ravel()
support_vectors_idx2 = (t * (X.dot(w2) + b2) < 1).ravel()
svm_clf1.support_vectors_ = X[support_vectors_idx1]
svm_clf2.support_vectors_ = X[support_vectors_idx2]

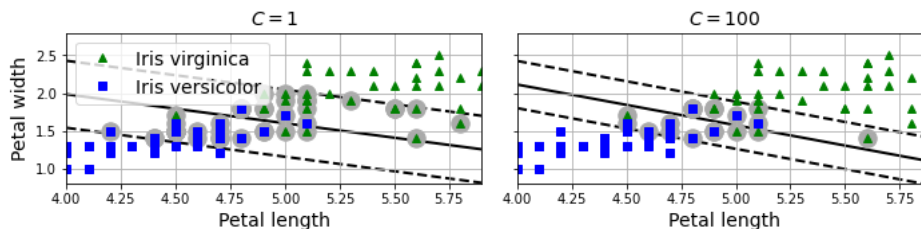
fig, axes = plt.subplots(ncols=2, figsize=(10, 2.7), sharey=True)

plt.sca(axes[0])
plt.plot(X[:, 0][y==1], X[:, 1][y==1], "g^", label="Iris virginica")
plt.plot(X[:, 0][y==0], X[:, 1][y==0], "bs", label="Iris versicolor")
plot_svc_decision_boundary(svm_clf1, 4, 5.9)
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.legend(loc="upper left")
plt.title(f"$C = {svm_clf1.C}$")
plt.axis([4, 5.9, 0.8, 2.8])
plt.grid()

plt.sca(axes[1])
plt.plot(X[:, 0][y==1], X[:, 1][y==1], "g^")
plt.plot(X[:, 0][y==0], X[:, 1][y==0], "bs")
plot_svc_decision_boundary(svm_clf2, 4, 5.99)
plt.xlabel("Petal length")
plt.title(f"$C = {svm_clf2.C}$")
plt.axis([4, 5.9, 0.8, 2.8])
plt.grid()

save_fig("regularization_plot")
plt.show()

```



✎ Phân loại SVM Phi tuyến (Non-linear SVM Classification)

extra code - this cell generates and saves Figure 5-5

```

X1D = np.linspace(-4, 4, 9).reshape(-1, 1)
X2D = np.c_[X1D, X1D**2]
y = np.array([0, 0, 1, 1, 1, 1, 1, 0, 0])

plt.figure(figsize=(10, 3))

plt.subplot(121)
plt.grid(True)
plt.axhline(y=0, color='k')
plt.plot(X1D[:, 0][y==0], np.zeros(4), "bs")
plt.plot(X1D[:, 0][y==1], np.zeros(5), "g^")
plt.gca().get_yaxis().set_ticks([])
plt.xlabel("$x_1$")
plt.axis([-4.5, 4.5, -0.2, 0.2])

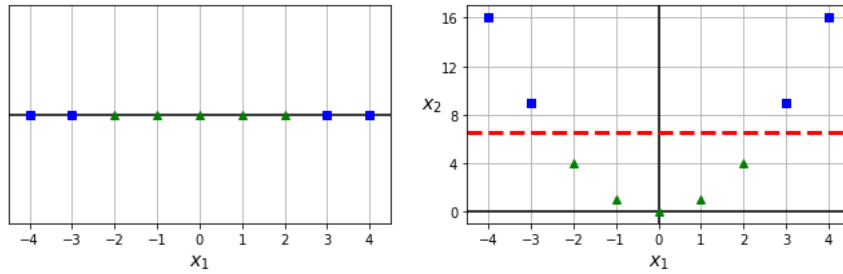
plt.subplot(122)
plt.grid(True)
plt.axhline(y=0, color='k')
plt.axvline(x=0, color='k')
plt.plot(X2D[:, 0][y==0], X2D[:, 1][y==0], "bs")
plt.plot(X2D[:, 0][y==1], X2D[:, 1][y==1], "g^")
plt.xlabel("$x_1$")
plt.ylabel("$x_2$", rotation=0)
plt.gca().get_yaxis().set_ticks([0, 4, 8, 12, 16])

```

```
plt.plot([-4.5, 4.5], [6.5, 6.5], "r--", linewidth=3)
plt.axis([-4.5, 4.5, -1, 17])

plt.subplots_adjust(right=1)

save_fig("higher_dimensions_plot", tight_layout=False)
plt.show()
```



```
from sklearn.datasets import make_moons
from sklearn.preprocessing import PolynomialFeatures

X, y = make_moons(n_samples=100, noise=0.15, random_state=42)

polynomial_svm_clf = make_pipeline(
    PolynomialFeatures(degree=3),
    StandardScaler(),
    LinearSVC(C=10, max_iter=10_000, dual=True, random_state=42)
)
polynomial_svm_clf.fit(X, y)

Pipeline(steps=[('polynomialfeatures', PolynomialFeatures(degree=3)),
                 ('standardscaler', StandardScaler()),
                 ('linearsvc',
                  LinearSVC(C=10, max_iter=10000, random_state=42))])
```

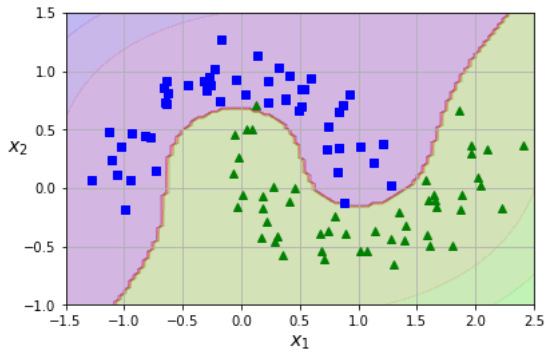
extra code - this cell generates and saves Figure 5-6

```
def plot_dataset(X, y, axes):
    plt.plot(X[:, 0][y==0], X[:, 1][y==0], "bs")
    plt.plot(X[:, 0][y==1], X[:, 1][y==1], "g^")
    plt.axis(axes)
    plt.grid(True)
    plt.xlabel("$x_1$")
    plt.ylabel("$x_2$", rotation=0)

def plot_predictions(clf, axes):
    x0s = np.linspace(axes[0], axes[1], 100)
    x1s = np.linspace(axes[2], axes[3], 100)
    x0, x1 = np.meshgrid(x0s, x1s)
    X = np.c_[x0.ravel(), x1.ravel()]
    y_pred = clf.predict(X).reshape(x0.shape)
    y_decision = clf.decision_function(X).reshape(x0.shape)
    plt.contourf(x0, x1, y_pred, cmap=plt.cm.brg, alpha=0.2)
    plt.contourf(x0, x1, y_decision, cmap=plt.cm.brg, alpha=0.1)

plot_predictions(polynomial_svm_clf, [-1.5, 2.5, -1, 1.5])
plot_dataset(X, y, [-1.5, 2.5, -1, 1.5])

save_fig("moons_polynomial_svc_plot")
plt.show()
```



✧ Hạt nhân Đa thức (Polynomial Kernel)

```
from sklearn.svm import SVC

poly_kernel_svm_clf = make_pipeline(StandardScaler(),
                                     SVC(kernel="poly", degree=3, coef0=1, C=5))
poly_kernel_svm_clf.fit(X, y)

Pipeline(steps=[('standardscaler', StandardScaler()),
                 ('svc', SVC(C=5, coef0=1, kernel='poly'))])
```

```
# extra code - this cell generates and saves Figure 5-7

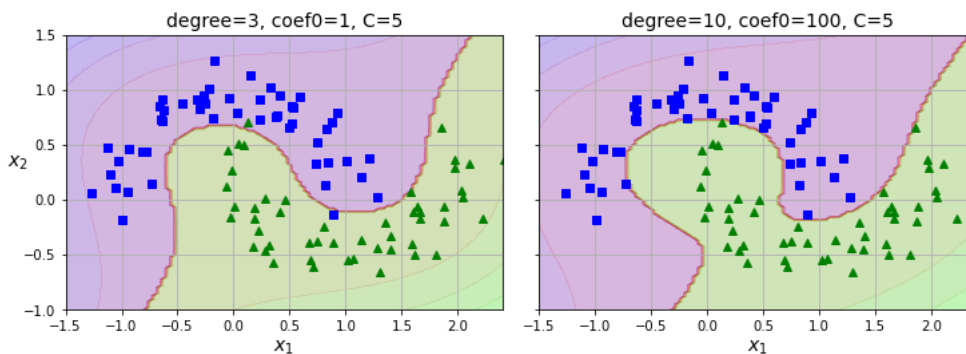
poly100_kernel_svm_clf = make_pipeline(
    StandardScaler(),
    SVC(kernel="poly", degree=10, coef0=100, C=5)
)
poly100_kernel_svm_clf.fit(X, y)

fig, axes = plt.subplots(ncols=2, figsize=(10.5, 4), sharey=True)

plt.sca(axes[0])
plot_predictions(poly_kernel_svm_clf, [-1.5, 2.45, -1, 1.5])
plot_dataset(X, y, [-1.5, 2.4, -1, 1.5])
plt.title("degree=3, coef0=1, C=5")

plt.sca(axes[1])
plot_predictions(poly100_kernel_svm_clf, [-1.5, 2.45, -1, 1.5])
plot_dataset(X, y, [-1.5, 2.4, -1, 1.5])
plt.title("degree=10, coef0=100, C=5")
plt.ylabel("")

save_fig("moons_kernelized_polynomial_svc_plot")
plt.show()
```



✧ Đặc trưng Tương đồng (Similarity Features)

```
# extra code - this cell generates and saves Figure 5-8

def gaussian_rbf(x, landmark, gamma):
    return np.exp(-gamma * np.linalg.norm(x - landmark, axis=1)**2)
```

```

gamma = 0.3

x1s = np.linspace(-4.5, 4.5, 200).reshape(-1, 1)
x2s = gaussian_rbf(x1s, -2, gamma)
x3s = gaussian_rbf(x1s, 1, gamma)

XK = np.c_[gaussian_rbf(X1D, -2, gamma), gaussian_rbf(X1D, 1, gamma)]
yk = np.array([0, 0, 1, 1, 1, 1, 1, 0, 0])

plt.figure(figsize=(10.5, 4))

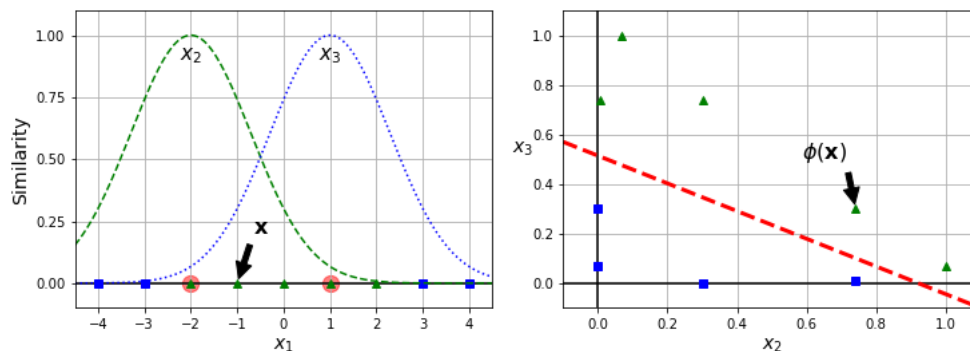
plt.subplot(121)
plt.grid(True)
plt.axhline(y=0, color='k')
plt.scatter(x=[-2, 1], y=[0, 0], s=150, alpha=0.5, c="red")
plt.plot(X1D[:, 0][yk==0], np.zeros(4), "bs")
plt.plot(X1D[:, 0][yk==1], np.zeros(5), "g^")
plt.plot(x1s, x2s, "g--")
plt.plot(x1s, x3s, "b:")
plt.gca().get_yaxis().set_ticks([0, 0.25, 0.5, 0.75, 1])
plt.xlabel("$x_1$")
plt.ylabel("Similarity")
plt.annotate(
    r'$\mathbf{x}$',
    xy=(X1D[3, 0], 0),
    xytext=(-0.5, 0.20),
    ha="center",
    arrowprops=dict(facecolor='black', shrink=0.1),
    fontsize=16,
)
plt.text(-2, 0.9, "$x_2$", ha="center", fontsize=15)
plt.text(1, 0.9, "$x_3$", ha="center", fontsize=15)
plt.axis([-4.5, 4.5, -0.1, 1.1])

plt.subplot(122)
plt.grid(True)
plt.axhline(y=0, color='k')
plt.axvline(x=0, color='k')
plt.plot(XK[:, 0][yk==0], XK[:, 1][yk==0], "bs")
plt.plot(XK[:, 0][yk==1], XK[:, 1][yk==1], "g^")
plt.xlabel("$x_2$")
plt.ylabel("$x_3$", rotation=0)
plt.annotate(
    r'$\phi(\mathbf{x})$',
    xy=(XK[3, 0], XK[3, 1]),
    xytext=(0.65, 0.50),
    ha="center",
    arrowprops=dict(facecolor='black', shrink=0.1),
    fontsize=16,
)
plt.plot([-0.1, 1.1], [0.57, -0.1], "r--", linewidth=3)
plt.axis([-0.1, 1.1, -0.1, 1.1])

plt.subplots_adjust(right=1)

save_fig("kernel_method_plot")
plt.show()

```



✧ Hạt nhân Gaussian RBF

```
rbf_kernel_svm_clf = make_pipeline(StandardScaler(),
                                   SVC(kernel="rbf", gamma=5, C=0.001))
rbf_kernel_svm_clf.fit(X, y)

Pipeline(steps=[('standardscaler', StandardScaler()),
                ('svc', SVC(C=0.001, gamma=5))])
```

extra code - this cell generates and saves Figure 5-9

```
from sklearn.svm import SVC

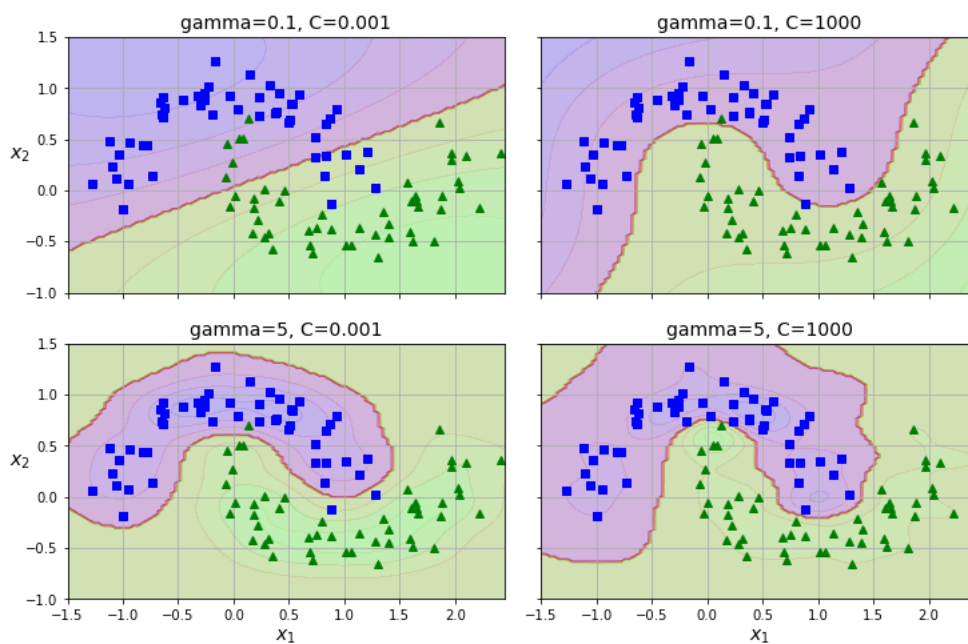
gamma1, gamma2 = 0.1, 5
C1, C2 = 0.001, 1000
hyperparams = (gamma1, C1), (gamma1, C2), (gamma2, C1), (gamma2, C2)

svm_clfs = []
for gamma, C in hyperparams:
    rbf_kernel_svm_clf = make_pipeline(
        StandardScaler(),
        SVC(kernel="rbf", gamma=gamma, C=C)
    )
    rbf_kernel_svm_clf.fit(X, y)
    svm_clfs.append(rbf_kernel_svm_clf)

fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(10.5, 7), sharex=True, sharey=True)

for i, svm_clf in enumerate(svm_clfs):
    plt.sca(axes[i // 2, i % 2])
    plot_predictions(svm_clf, [-1.5, 2.45, -1, 1.5])
    plot_dataset(X, y, [-1.5, 2.45, -1, 1.5])
    gamma, C = hyperparams[i]
    plt.title(f"gamma={gamma}, C={C}")
    if i in (0, 1):
        plt.xlabel("")
    if i in (1, 3):
        plt.ylabel("")

save_fig("moons_rbf_svc_plot")
plt.show()
```



✧ Hồi quy SVM (SVM Regression)

```

from sklearn.svm import LinearSVR

# extra code - these 3 lines generate a simple linear dataset
np.random.seed(42)
X = 2 * np.random.rand(50, 1)
y = 4 + 3 * X[:, 0] + np.random.randn(50)

svm_reg = make_pipeline(StandardScaler(),
                        LinearSVR(epsilon=0.5, dual=True, random_state=42))
svm_reg.fit(X, y)

```

```

Pipeline(steps=[('standardscaler', StandardScaler()),
                ('linearsvr', LinearSVR(epsilon=0.5, random_state=42))])

```

```

# extra code - this cell generates and saves Figure 5-10

def find_support_vectors(svm_reg, X, y):
    y_pred = svm_reg.predict(X)
    epsilon = svm_reg[-1].epsilon
    off_margin = np.abs(y - y_pred) >= epsilon
    return np.argwhere(off_margin)

def plot_svm_regression(svm_reg, X, y, axes):
    x1s = np.linspace(axes[0], axes[1], 100).reshape(100, 1)
    y_pred = svm_reg.predict(x1s)
    epsilon = svm_reg[-1].epsilon
    plt.plot(x1s, y_pred, "k-", linewidth=2, label=r"$\hat{y}$", zorder=-2)
    plt.plot(x1s, y_pred + epsilon, "k--", zorder=-2)
    plt.plot(x1s, y_pred - epsilon, "k--", zorder=-2)
    plt.scatter(X[svm_reg._support], y[svm_reg._support], s=180,
                facecolors='#AAA', zorder=-1)
    plt.plot(X, y, "bo")
    plt.xlabel("$x_1$")
    plt.legend(loc="upper left")
    plt.axis(axes)

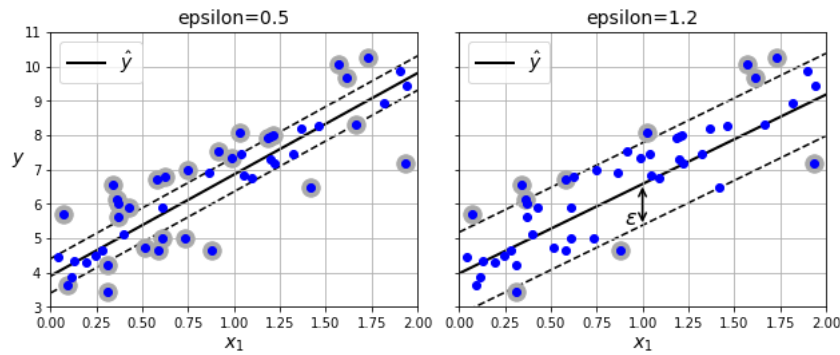
svm_reg2 = make_pipeline(StandardScaler(),
                        LinearSVR(epsilon=1.2, dual=True, random_state=42))
svm_reg2.fit(X, y)

svm_reg._support = find_support_vectors(svm_reg, X, y)
svm_reg2._support = find_support_vectors(svm_reg2, X, y)

eps_x1 = 1
eps_y_pred = svm_reg2.predict([[eps_x1]])

fig, axes = plt.subplots(ncols=2, figsize=(9, 4), sharey=True)
plt.sca(axes[0])
plot_svm_regression(svm_reg, X, y, [0, 2, 3, 11])
plt.title(f"epsilon={svm_reg[-1].epsilon}")
plt.ylabel("$y$", rotation=0)
plt.grid()
plt.sca(axes[1])
plot_svm_regression(svm_reg2, X, y, [0, 2, 3, 11])
plt.title(f"epsilon={svm_reg2[-1].epsilon}")
plt.annotate(
    '', xy=(eps_x1, eps_y_pred), xycoords='data',
    xytext=(eps_x1, eps_y_pred - svm_reg2[-1].epsilon),
    textcoords='data', arrowprops={'arrowstyle': '<->', 'linewidth': 1.5}
)
plt.text(0.90, 5.4, r"$\epsilon$", fontsize=16)
plt.grid()
save_fig("svm_regression_plot")
plt.show()

```



```
from sklearn.svm import SVR

# extra code - these 3 lines generate a simple quadratic dataset
np.random.seed(42)
X = 2 * np.random.rand(50, 1) - 1
y = 0.2 + 0.1 * X[:, 0] + 0.5 * X[:, 0] ** 2 + np.random.randn(50) / 10

svm_poly_reg = make_pipeline(StandardScaler(),
                             SVR(kernel="poly", degree=2, C=0.01, epsilon=0.1))
svm_poly_reg.fit(X, y)

Pipeline(steps=[('standardscaler', StandardScaler()),
                 ('svr', SVR(C=0.01, degree=2, kernel='poly'))])
```

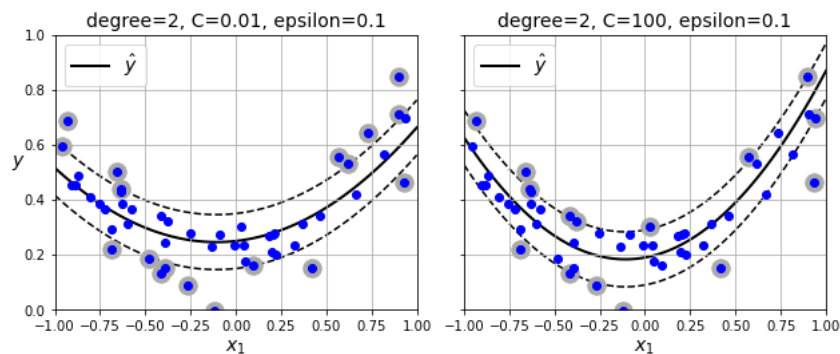
```
# extra code - this cell generates and saves Figure 5-11

svm_poly_reg2 = make_pipeline(StandardScaler(),
                              SVR(kernel="poly", degree=2, C=100))
svm_poly_reg2.fit(X, y)

svm_poly_reg._support = find_support_vectors(svm_poly_reg, X, y)
svm_poly_reg2._support = find_support_vectors(svm_poly_reg2, X, y)

fig, axes = plt.subplots(ncols=2, figsize=(9, 4), sharey=True)
plt.sca(axes[0])
plot_svm_regression(svm_poly_reg, X, y, [-1, 1, 0, 1])
plt.title(f"degree={svm_poly_reg[-1].degree}, "
          f"C={svm_poly_reg[-1].C}, "
          f"epsilon={svm_poly_reg[-1].epsilon}")
plt.ylabel("$y$", rotation=0)
plt.grid()

plt.sca(axes[1])
plot_svm_regression(svm_poly_reg2, X, y, [-1, 1, 0, 1])
plt.title(f"degree={svm_poly_reg2[-1].degree}, "
          f"C={svm_poly_reg2[-1].C}, "
          f"epsilon={svm_poly_reg2[-1].epsilon}")
plt.grid()
save_fig("svm_with_polynomial_kernel_plot")
plt.show()
```



✧ Tìm hiểu sâu hơn (Under the hood)

extra code - this cell generates and saves Figure 5-12

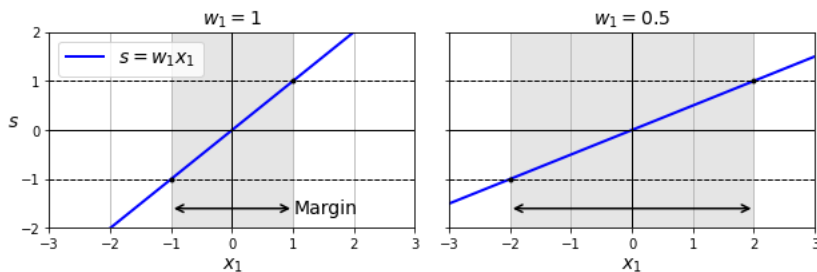
```
import matplotlib.patches as patches
```

```
def plot_2D_decision_function(w, b, ylabel=True, x1_lim=[-3, 3]):
    x1 = np.linspace(x1_lim[0], x1_lim[1], 200)
    y = w * x1 + b
    half_margin = 1 / w

    plt.plot(x1, y, "b-", linewidth=2, label=r"$s = w_1 x_1$")
    plt.axhline(y=0, color='k', linewidth=1)
    plt.axvline(x=0, color='k', linewidth=1)
    rect = patches.Rectangle((-half_margin, -2), 2 * half_margin, 4,
                             edgecolor='none', facecolor='gray', alpha=0.2)
    plt.gca().add_patch(rect)
    plt.plot([-3, 3], [1, 1], "k--", linewidth=1)
    plt.plot([-3, 3], [-1, -1], "k--", linewidth=1)
    plt.plot(half_margin, 1, "k.")
    plt.plot(-half_margin, -1, "k.")
    plt.axis(x1_lim + [-2, 2])
    plt.xlabel("$x_1$")
    if ylabel:
        plt.ylabel("$s$", rotation=0, labelpad=5)
        plt.legend()
        plt.text(1.02, -1.6, "Margin", ha="left", va="center", color="k")

    plt.annotate(
        '', xy=(-half_margin, -1.6), xytext=(half_margin, -1.6),
        arrowprops={'ec': 'k', 'arrowstyle': '<->', 'linewidth': 1.5}
    )
    plt.title(f"$w_1 = {w}$")
```

```
fig, axes = plt.subplots(ncols=2, figsize=(9, 3.2), sharey=True)
plt.sca(axes[0])
plot_2D_decision_function(1, 0)
plt.grid()
plt.sca(axes[1])
plot_2D_decision_function(0.5, 0, ylabel=False)
plt.grid()
save_fig("small_w_large_margin_plot")
plt.show()
```



extra code - this cell generates and saves Figure 5-13

```
s = np.linspace(-2.5, 2.5, 200)
hinge_pos = np.where(1 - s < 0, 0, 1 - s) # max(0, 1 - s)
hinge_neg = np.where(1 + s < 0, 0, 1 + s) # max(0, 1 + s)

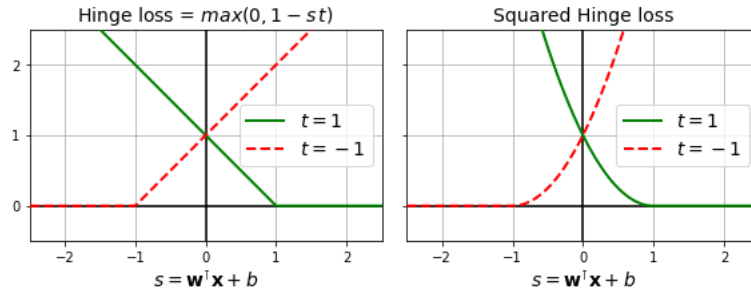
titles = (r"Hinge loss = $\max(0, 1 - s)$", "Squared Hinge loss")

fig, axs = plt.subplots(1, 2, sharey=True, figsize=(8.2, 3))

for ax, loss_pos, loss_neg, title in zip(
    axs, (hinge_pos, hinge_pos ** 2), (hinge_neg, hinge_neg ** 2), titles):
    ax.plot(s, loss_pos, "g-", linewidth=2, zorder=10, label="$t=1$")
    ax.plot(s, loss_neg, "r--", linewidth=2, zorder=10, label="$t=-1$")
    ax.grid(True)
    ax.axhline(y=0, color='k')
    ax.axvline(x=0, color='k')
    ax.set_xlabel(r"$s = \mathbf{w}^T \mathbf{x} + b$")
    ax.axis([-2.5, 2.5, -0.5, 2.5])
    ax.legend(loc="center right")
    ax.set_title(title)
    ax.set_yticks(np.arange(0, 2.5, 1))
```

```
ax.set_aspect("equal")

save_fig("hinge_plot")
plt.show()
```



✎ Tài liệu bổ sung

✎ Triển khai Linear SVM Classifier bằng Batch Gradient Descent

```
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 2)
```

```
from sklearn.base import BaseEstimator

class MyLinearSVC(BaseEstimator):
    def __init__(self, C=1, eta0=1, eta_d=10000, n_epochs=1000,
                 random_state=None):
        self.C = C
        self.eta0 = eta0
        self.n_epochs = n_epochs
        self.random_state = random_state
        self.eta_d = eta_d

    def eta(self, epoch):
        return self.eta0 / (epoch + self.eta_d)

    def fit(self, X, y):
        # Random initialization
        if self.random_state:
            np.random.seed(self.random_state)
        w = np.random.randn(X.shape[1], 1) # n feature weights
        b = 0

        t = np.array(y, dtype=np.float64).reshape(-1, 1) * 2 - 1
        X_t = X * t
        self.Js = []

        # Training
        for epoch in range(self.n_epochs):
            support_vectors_idx = (X_t.dot(w) + t * b < 1).ravel()
            X_t_sv = X_t[support_vectors_idx]
            t_sv = t[support_vectors_idx]

            J = 1/2 * (w * w).sum() + self.C * ((1 - X_t_sv.dot(w)).sum() - b * t_sv.sum())
            self.Js.append(J)

            w_gradient_vector = w - self.C * X_t_sv.sum(axis=0).reshape(-1, 1)
            b_derivative = -self.C * t_sv.sum()

            w = w - self.eta(epoch) * w_gradient_vector
            b = b - self.eta(epoch) * b_derivative

        self.intercept_ = np.array([b])
        self.coef_ = np.array([w])
        support_vectors_idx = (X_t.dot(w) + t * b < 1).ravel()
        self.support_vectors_ = X[support_vectors_idx]
        return self
```

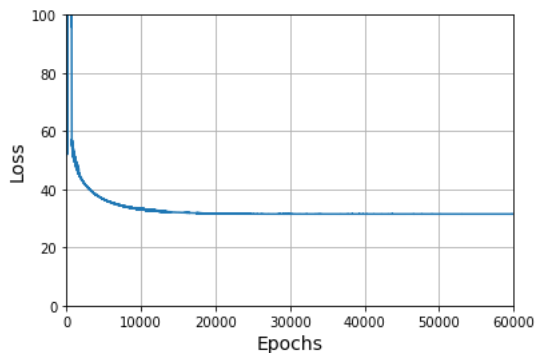
```
def decision_function(self, X):
    return X.dot(self.coef_[0]) + self.intercept_[0]

def predict(self, X):
    return self.decision_function(X) >= 0
```

```
C = 2
svm_clf = MyLinearSVC(C=C, eta0 = 10, eta_d = 1000, n_epochs=60000,
                      random_state=2)
svm_clf.fit(X, y)
svm_clf.predict(np.array([[5, 2], [4, 1]]))
```

```
array([[ True],
       [False]])
```

```
plt.plot(range(svm_clf.n_epochs), svm_clf.Js)
plt.axis([0, svm_clf.n_epochs, 0, 100])
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.grid()
plt.show()
```



```
print(svm_clf.intercept_, svm_clf.coef_)
```

```
[-15.56761653] [[2.28120287]
 [2.71621742]]
```

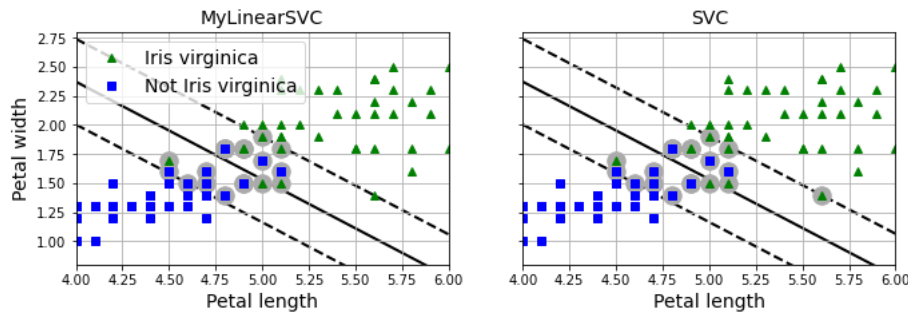
```
svm_clf2 = SVC(kernel="linear", C=C)
svm_clf2.fit(X, y.ravel())
print(svm_clf2.intercept_, svm_clf2.coef_)
```

```
[-15.51721253] [[2.27128546 2.71287145]]
```

```
yr = y.ravel()
fig, axes = plt.subplots(ncols=2, figsize=(11, 3.2), sharey=True)
plt.sca(axes[0])
plt.plot(X[:, 0][yr==1], X[:, 1][yr==1], "g^", label="Iris virginica")
plt.plot(X[:, 0][yr==0], X[:, 1][yr==0], "bs", label="Not Iris virginica")
plot_svc_decision_boundary(svm_clf, 4, 6)
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.title("MyLinearSVC")
plt.axis([4, 6, 0.8, 2.8])
plt.legend(loc="upper left")
plt.grid()

plt.sca(axes[1])
plt.plot(X[:, 0][yr==1], X[:, 1][yr==1], "g^")
plt.plot(X[:, 0][yr==0], X[:, 1][yr==0], "bs")
plot_svc_decision_boundary(svm_clf2, 4, 6)
plt.xlabel("Petal length")
plt.title("SVC")
plt.axis([4, 6, 0.8, 2.8])
plt.grid()

plt.show()
```



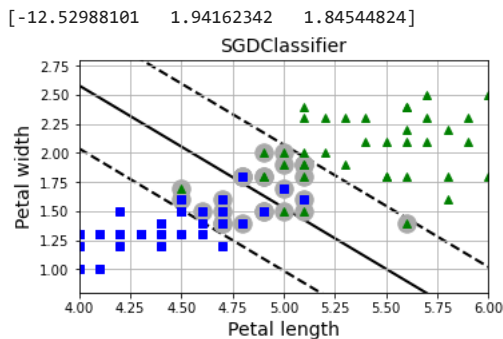
```
from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(loss="hinge", alpha=0.017, max_iter=1000, tol=1e-3,
                        random_state=42)
sgd_clf.fit(X, y)

m = len(X)
t = np.array(y).reshape(-1, 1) * 2 - 1 # -1 if y == 0, or +1 if y == 1
X_b = np.c_[np.ones((m, 1)), X] # Add bias input x0=1
X_b_t = X_b * t
sgd_theta = np.r_[sgd_clf.intercept_[0], sgd_clf.coef_[0]]
print(sgd_theta)
support_vectors_idx = (X_b_t.dot(sgd_theta) < 1).ravel()
sgd_clf.support_vectors_ = X[support_vectors_idx]
sgd_clf.C = C

plt.figure(figsize=(5.5, 3.2))
plt.plot(X[:, 0][yr==1], X[:, 1][yr==1], "g^")
plt.plot(X[:, 0][yr==0], X[:, 1][yr==0], "bs")
plot_svc_decision_boundary(sgd_clf, 4, 6)
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.title("SGDClassifier")
plt.axis([4, 6, 0.8, 2.8])
plt.grid()

plt.show()
```



✧ Giải bài tập

✧ Từ câu 1 đến câu 8

1. Ý tưởng cơ bản đằng sau Support Vector Machines là khớp với một "con đường" rộng nhất có thể giữa các lớp. Mục tiêu là có biên (margin) lớn nhất giữa ranh giới quyết định và các thực thể huấn luyện. Trong Soft Margin Classification, SVM tìm kiếm sự cân bằng giữa việc phân loại đúng hoàn hảo và việc giữ cho biên rộng nhất có thể. Các ý tưởng quan trọng khác bao gồm sử dụng hạt nhân (kernel) cho dữ liệu phi tuyến.
2. Vectơ hỗ trợ (support vector) là các thực thể nằm trên biên (bao gồm cả các điểm nằm trên đường biên). Ranh giới quyết định được xác định hoàn toàn bởi các vectơ này. Các điểm không phải là vectơ hỗ trợ sẽ không ảnh hưởng đến mô hình.
3. SVM cố gắng tìm biên lớn nhất, do đó nếu dữ liệu không được chuẩn hóa tỷ lệ (scaled), SVM sẽ có xu hướng bỏ qua các đặc trưng có giá trị nhỏ.

4. Bạn có thể sử dụng `decision_function()` để lấy điểm tin cậy (khoảng cách tới ranh giới), nhưng nó không phải là xác suất trực tiếp. Cần đặt `probability=True` trong `SVC` để ước tính xác suất qua cross-validation.
5. `SVC`, `LinearSVC`, và `SGDClassifier` đều dùng cho phân loại tuyến tính. `SVC` hỗ trợ kernel phi tuyến nhưng không scale tốt với tập dữ liệu lớn. `LinearSVC` tối ưu cho tuyến tính. `SGDClassifier` linh hoạt và hỗ trợ học online.
6. Nếu RBF SVM bị underfit, có thể do quá nhiều chính quy hóa (regularization). Hãy thử tăng `gamma` hoặc `C` (hoặc cả hai).
7. Mô hình SVM Regression cố gắng khớp nhiều thực thể nhất có thể vào một biên nhỏ quanh dự đoán. Nó được gọi là epsilon-insensitive vì các điểm nằm trong biên này không ảnh hưởng đến mô hình.
8. Thủ thuật hạt nhân (Kernel trick) giúp huấn luyện mô hình SVM phi tuyến tương đương với việc chiếu dữ liệu lên không gian chiều cao hơn mà không cần thực hiện phép tính chiếu đó một cách tường minh.

▼ Bài 9

Bài tập: Huấn luyện một `LinearSVC` trên một tập dữ liệu có thể phân tách tuyến tính. Sau đó, huấn luyện `SVC` và `SGDClassifier` trên cùng một tập dữ liệu để xem liệu bạn có thể tạo ra các mô hình gần như giống hệt nhau hay không.

Hãy sử dụng tập dữ liệu Iris: các lớp Iris Setosa và Iris Versicolor có thể phân tách tuyến tính.

```
from sklearn import datasets

iris = datasets.load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris.target

setosa_or_versicolor = (y == 0) | (y == 1)
X = X[setosa_or_versicolor]
y = y[setosa_or_versicolor]
```

Bây giờ hãy xây dựng và huấn luyện 3 mô hình:

- `LinearSVC` sử dụng `loss="squared_hinge"` mặc định, nên ta cần đặt `loss="hinge"` để tương thích.
- `SVC` dùng kernel RBF mặc định, nên ta đặt `kernel="linear"`.
- `SGDClassifier` không có tham số `C`, nhưng có `alpha` để điều chỉnh chính quy hóa.

```
from sklearn.svm import SVC, LinearSVC
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import StandardScaler

C = 5
alpha = 0.05

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

lin_clf = LinearSVC(loss="hinge", C=C, dual=True, random_state=42).fit(X_scaled, y)
svc_clf = SVC(kernel="linear", C=C).fit(X_scaled, y)
sgd_clf = SGDClassifier(alpha=alpha, random_state=42).fit(X_scaled, y)
```

Hãy vẽ ranh giới quyết định của ba mô hình này:

```
def compute_decision_boundary(model):
    w = -model.coef_[0, 0] / model.coef_[0, 1]
    b = -model.intercept_[0] / model.coef_[0, 1]
    return scaler.inverse_transform([[-10, -10 * w + b], [10, 10 * w + b]])

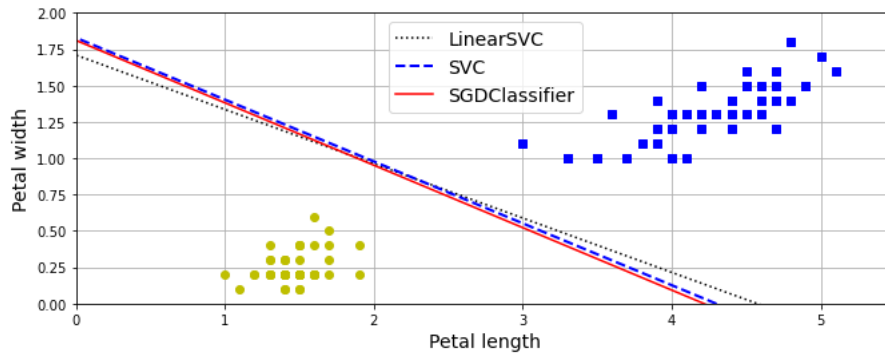
lin_line = compute_decision_boundary(lin_clf)
svc_line = compute_decision_boundary(svc_clf)
sgd_line = compute_decision_boundary(sgd_clf)

# Plot all three decision boundaries
plt.figure(figsize=(11, 4))
plt.plot(lin_line[:, 0], lin_line[:, 1], "k:", label="LinearSVC")
plt.plot(svc_line[:, 0], svc_line[:, 1], "b--", linewidth=2, label="SVC")
plt.plot(sgd_line[:, 0], sgd_line[:, 1], "r-", label="SGDClassifier")
plt.plot(X[:, 0][y==1], X[:, 1][y==1], "bs") # label="Iris versicolor"
plt.plot(X[:, 0][y==0], X[:, 1][y==0], "yo") # label="Iris setosa"
```



```
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.legend(loc="upper center")
plt.axis([0, 5.5, 0, 2])
plt.grid()
```

```
plt.show()
```



Kết quả khá tương đồng!

✶ Bài 10

Bài tập: Huấn luyện bộ phân loại SVM trên tập dữ liệu Wine. Mục tiêu là dự đoán người trồng rượu dựa trên phân tích hóa học. Vì SVM là bộ phân loại nhị phân, bạn cần chiến lược One-vs-All (OvR).

Đầu tiên, tải dữ liệu, xem mô tả và chia tập huấn luyện/kiểm tra:

```
from sklearn.datasets import load_wine

wine = load_wine(as_frame=True)
```

```
print(wine.DESCR)
```

```
.. _wine_dataset:
```

```
Wine recognition dataset
```

```
-----
```

```
**Data Set Characteristics:**
```

```
:Number of Instances: 178 (50 in each of three classes)
:Number of Attributes: 13 numeric, predictive attributes and the class
:Attribute Information:
  - Alcohol
  - Malic acid
  - Ash
  - Alcalinity of ash
  - Magnesium
  - Total phenols
  - Flavanoids
  - Nonflavanoid phenols
  - Proanthocyanins
  - Color intensity
  - Hue
  - OD280/OD315 of diluted wines
  - Proline

- class:
  - class_0
  - class_1
  - class_2
```

```
:Summary Statistics:
```

```
=====
              Min    Max    Mean    SD
=====
Alcohol:      11.0  14.8   13.0   0.8
```

```
<<26 more lines>>
```

wine.

Original Owners:

Forina, M. et al, PARVUS -
An Extendible Package for Data Exploration, Classification and Correlation.
Institute of Pharmaceutical and Food Analysis and Technologies,
Via Brigata Salerno, 16147 Genoa, Italy.

Citation:

Lichman, M. (2013). UCI Machine Learning Repository
[<https://archive.ics.uci.edu/ml>]. Irvine, CA: University of California,
School of Information and Computer Science.

.. topic:: References

(1) S. Aeberhard, D. Coomans and O. de Vel,
Comparison of Classifiers in High Dimensional Settings,
Tech. Rep. no. 92-02, (1992), Dept. of Computer Science and Dept. of
Mathematics and Statistics, James Cook University of North Queensland.
(Also submitted to Technometrics).

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    wine.data, wine.target, random_state=42)
```

X_train.head()

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	c
2	13.16	2.36	2.67		18.6	101.0	2.80	3.24	0.30	2.81
100	12.08	2.08	1.70		17.5	97.0	2.23	2.17	0.26	1.40
122	12.42	4.43	2.73		26.5	102.0	2.20	2.13	0.43	1.71
154	12.58	1.29	2.10		20.0	103.0	1.48	0.58	0.53	1.40
51	13.83	1.65	2.60		17.2	94.0	2.45	2.99	0.22	2.29

y_train.head()

```
2      0  
100    1  
122    1  
154    2  
51      0  
Name: target, dtype: int64
```

Bắt đầu với `LinearSVC` tuyến tính. Nó sẽ tự động sử dụng chiến lược OvR.

```
lin_clf = LinearSVC(dual=True, random_state=42)  
lin_clf.fit(X_train, y_train)
```

```
/Users/ageron/miniconda3/envs/homl3/lib/python3.8/site-packages/sklearn/svm/_base.py:1206: ConvergenceWarning: Liblinear failed  
warnings.warn(  
LinearSVC(random_state=42)
```

Mô hình không hội tụ. Hãy thử tăng số lượng vòng lặp `max_iter`.

```
lin_clf = LinearSVC(max_iter=1_000_000, dual=True, random_state=42)  
lin_clf.fit(X_train, y_train)
```

```
/Users/ageron/miniconda3/envs/homl3/lib/python3.8/site-packages/sklearn/svm/_base.py:1206: ConvergenceWarning: Liblinear failed  
warnings.warn(  
LinearSVC(max_iter=1000000, random_state=42)
```

Vẫn không hội tụ sau 1 triệu vòng lặp. Hãy dùng `cross_val_score` để đánh giá baseline.

```
from sklearn.model_selection import cross_val_score
```

```
cross_val_score(lin_clf, X_train, y_train).mean()
```

```
/Users/ageron/miniconda3/envs/homl3/lib/python3.8/site-packages/sklearn/svm/_base.py:1206: ConvergenceWarning: Liblinear failed
warnings.warn(
/Users/ageron/miniconda3/envs/homl3/lib/python3.8/site-packages/sklearn/svm/_base.py:1206: ConvergenceWarning: Liblinear failed
warnings.warn(
/Users/ageron/miniconda3/envs/homl3/lib/python3.8/site-packages/sklearn/svm/_base.py:1206: ConvergenceWarning: Liblinear failed
warnings.warn(
/Users/ageron/miniconda3/envs/homl3/lib/python3.8/site-packages/sklearn/svm/_base.py:1206: ConvergenceWarning: Liblinear failed
warnings.warn(
/Users/ageron/miniconda3/envs/homl3/lib/python3.8/site-packages/sklearn/svm/_base.py:1206: ConvergenceWarning: Liblinear failed
warnings.warn(
0.90997150997151
```

Vấn đề nằm ở chỗ chúng ta quên chuẩn hóa đặc trưng (feature scaling). Hãy thực hiện nó:

```
lin_clf = make_pipeline(StandardScaler(),
                        LinearSVC(dual=True, random_state=42))
lin_clf.fit(X_train, y_train)
```

```
Pipeline(steps=[('standardscaler', StandardScaler()),
                 ('linearsvc', LinearSVC(random_state=42))])
```

Bây giờ mô hình đã hội tụ. Hãy đo lường hiệu suất:

```
from sklearn.model_selection import cross_val_score
```

```
cross_val_score(lin_clf, X_train, y_train).mean()
```

```
0.9774928774928775
```

Độ chính xác đạt 97.7%, rất tốt!

Thử nghiệm với kernel phi tuyến bằng `SVC` mặc định:

```
svm_clf = make_pipeline(StandardScaler(), SVC(random_state=42))
cross_val_score(svm_clf, X_train, y_train).mean()
```

```
0.9698005698005698
```

Chưa tốt hơn, cần tinh chỉnh siêu tham số bằng `RandomizedSearchCV`:

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform, uniform

param_distrib = {
    "svc__gamma": loguniform(0.001, 0.1),
    "svc__C": uniform(1, 10)
}
rnd_search_cv = RandomizedSearchCV(svm_clf, param_distrib, n_iter=100, cv=5,
                                   random_state=42)
rnd_search_cv.fit(X_train, y_train)
rnd_search_cv.best_estimator_
```

```
Pipeline(steps=[('standardscaler', StandardScaler()),
                 ('svc',
                  SVC(C=9.925589984899778, gamma=0.011986281799901176,
                      random_state=42))])
```

```
rnd_search_cv.best_score_
```

```
0.9925925925925926
```

Kết quả tuyệt vời! Hãy kiểm tra trên tập test.

```
rnd_search_cv.score(X_test, y_test)
```

```
0.9777777777777777
```

SVM hạt nhân được điều chỉnh này hoạt động tốt hơn mô hình, nhưng chúng tôi nhận được điểm số trên tập thử nghiệm thấp hơn so với việc chúng tôi đo bằng cách sử dụng xác thực chéo. Điều này khá phổ biến: vì chúng tôi đã thực hiện quá nhiều điều chỉnh siêu tham số, chúng tôi đã kết thúc một chút quá khớp với các bộ kiểm tra xác thực chéo. Thật hấp dẫn để điều chỉnh các siêu tham số nhiều hơn một chút cho đến khi chúng ta nhận được kết quả tốt hơn trên tập thử nghiệm, nhưng điều này có lẽ sẽ không giúp ích gì, vì chúng ta sẽ bắt đầu quá phù hợp với tập thử nghiệm. Dù sao, điểm số này không tệ chút nào, vì vậy hãy dừng lại ở đây. LinearSVC

11.

▼ Bài 11

Bài tập: Huấn luyện và tinh chỉnh SVM Regressor trên tập dữ liệu California Housing. Do tập dữ liệu lớn (>20,000 thực thể), hãy dùng 2,000 mẫu để tìm siêu tham số.

Hãy tải tập dữ liệu:

```
from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing()
X = housing.data
y = housing.target
```

Chia nó thành một tập luyện và một tập thử nghiệm:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
```

Đừng quên mở rộng quy mô dữ liệu!

Trước tiên, hãy đào tạo một cách đơn giản: LinearSVR

```
from sklearn.svm import LinearSVR

lin_svr = make_pipeline(StandardScaler(), LinearSVR(dual=True, random_state=42))
lin_svr.fit(X_train, y_train)

/Users/ageron/miniconda3/envs/homl3/lib/python3.8/site-packages/sklearn/svm/_base.py:1206: ConvergenceWarning: Liblinear failed
warnings.warn(
Pipeline(steps=[('standardscaler', StandardScaler()),
                 ('linearsvr', LinearSVR(random_state=42))])
```

Nó không hội tụ, vì vậy hãy tăng :max_iter

```
lin_svr = make_pipeline(StandardScaler(),
                        LinearSVR(max_iter=5000, dual=True, random_state=42))
lin_svr.fit(X_train, y_train)

Pipeline(steps=[('standardscaler', StandardScaler()),
                 ('linearsvr', LinearSVR(max_iter=5000, random_state=42))])
```

Hãy xem nó hoạt động như thế nào trên bộ đào tạo:

```
from sklearn.metrics import mean_squared_error

y_pred = lin_svr.predict(X_train)
mse = mean_squared_error(y_train, y_pred)
mse

0.9595484665813285
```

Hãy xem xét RMSE:

```
np.sqrt(mse)
```

```
0.979565447829459
```

Trong bộ dữ liệu này, các mục tiêu đại diện cho hàng trăm nghìn đô la. RMSE đưa ra ý tưởng sơ bộ về loại lỗi bạn nên mong đợi (với trọng số cao hơn đối với các lỗi lớn): vì vậy với mô hình này, chúng ta có thể mong đợi lỗi gần 98.000 đô la! Không tuyệt vời. Hãy xem liệu chúng ta có thể làm tốt hơn với RBF Kernel hay không. Chúng tôi sẽ sử dụng tìm kiếm ngẫu nhiên với xác thực chéo để tìm các giá trị siêu tham số thích hợp cho γ :

```
from sklearn.svm import SVR
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform, uniform

svm_reg = make_pipeline(StandardScaler(), SVR())

param_distrib = {
    "svr__gamma": loguniform(0.001, 0.1),
    "svr__C": uniform(1, 10)
}
rnd_search_cv = RandomizedSearchCV(svm_reg, param_distrib,
                                   n_iter=100, cv=3, random_state=42)
rnd_search_cv.fit(X_train[:2000], y_train[:2000])
```

```
RandomizedSearchCV(cv=3,
                   estimator=Pipeline(steps=[('standardscaler',
                                              StandardScaler()),
                                              ('svr', SVR())]),
                   n_iter=100,
                   param_distributions={'svr__C': <scipy.stats._distn_infrastructure.rv_frozen object at 0x7ff030704ee0>,
                                       'svr__gamma': <scipy.stats._distn_infrastructure.rv_frozen object at 0x7ff030704fd0>},
                   random_state=42)
```

```
rnd_search_cv.best_estimator_
```

```
Pipeline(steps=[('standardscaler', StandardScaler()),
                 ('svr', SVR(C=4.63629602379294, gamma=0.08781408196485974))])
```

```
-cross_val_score(rnd_search_cv.best_estimator_, X_train, y_train,
                  scoring="neg_root_mean_squared_error")
```

```
array([0.58835648, 0.57468589, 0.58085278, 0.57109886, 0.59853029])
```